הודיה חנניאייב - 324857549

תהילה חנאייב – 325435956

<u>מערכות הפעלה תרגיל מעבדה 7</u>

זה כשהרצנו וזה נתקע בגלל קיפאון

```
tehila2003@DESKTOP-OPO0EU0: ~
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage


This message is shown once a day. To disable it please create the
/home/tehila2003/.hushlogin file.
tehila2003@DESKTOP-OPO0EU0:~$ nano dpp.c
tehila2003@DESKTOP-OPO0EU0:~$ gcc dpp.c -o dpp
tehila2003@DESKTOP-OPO0EU0:~$ ./dpp
Philosopher 0 will think for 2 seconds
Philosopher 2 will think for 2 seconds
Philosopher 1 will think for 2 seconds
Philosopher 3 will think for 2 seconds
Philosopher 4 will think for 2 seconds
Philosopher 0 is waiting to pick up chopstick 0
Philosopher 3 is waiting to pick up chopstick 3
Philosopher 2 is waiting to pick up chopstick 2
Philosopher 2 picked up chopstick 2
Philosopher 2 is waiting to pick up chopstick 3
Philosopher 4 is waiting to pick up chopstick 4
Philosopher 4 picked up chopstick 4
Philosopher 4 is waiting to pick up chopstick 0
Philosopher 1 is waiting to pick up chopstick 1
Philosopher 1 picked up chopstick 1
Philosopher 1 is waiting to pick up chopstick 2
Philosopher 0 picked up chopstick 0
Philosopher 0 is waiting to pick up chopstick 1
Philosopher 3 picked up chopstick 3
Philosopher 3 is waiting to pick up chopstick 4
^C
tehila2003@DESKTOP-OPO0EU0:~$ pico dpp.c
```

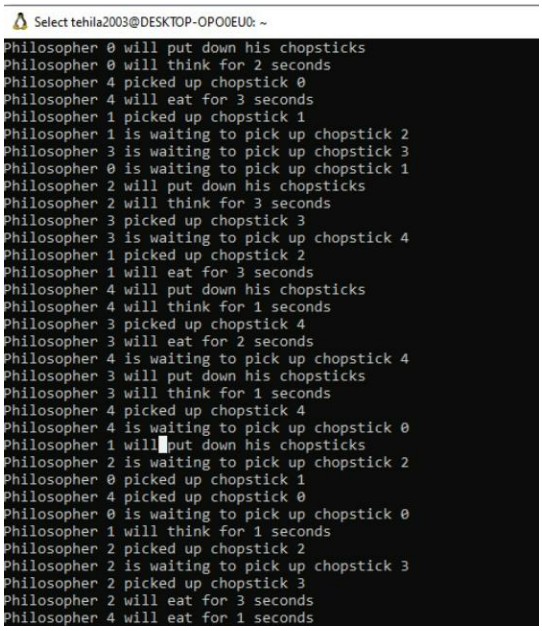
.1

נבצע תיקון על ידי שינוי ההתנהגות של הפילוסוף הראשון (פילוסוף מספר 0), כך שהוא ירים את המקלות בסדר הפוך מזה של שאר הפילוסופים. כלומר, במקום להרים קודם את המקל הימני ולאחר מכן את השמאלי, הוא ירים קודם את המקל השמאלי ולאחר מכן את הימני.

הנה הקוד המתוקן:

```
tehila2003@DESKTOP-OPO0EU0: ~
  GNU nano 6.2                                                    dpp.c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <time.h>
#include <stdlib.h>

#define NUMBER_OF_PHILOSOPHERS 5

void *philosopher(void *);
void think(int);
void pickUp(int);
void eat(int);
void putDown(int);

pthread_mutex_t chopsticks[NUMBER_OF_PHILOSOPHERS];
pthread_t philosophers[NUMBER_OF_PHILOSOPHERS];
pthread_attr_t attributes[NUMBER_OF_PHILOSOPHERS];

int main() {
    long i;
    srand(time(NULL));
    for (i = 0; i < NUMBER_OF_PHILOSOPHERS; ++i) {
        pthread_mutex_init(&chopsticks[i], NULL);
    }

    for (i = 0; i < NUMBER_OF_PHILOSOPHERS; ++i) {
        pthread_attr_init(&attributes[i]);
    }

    for (i = 0; i < NUMBER_OF_PHILOSOPHERS; ++i) {
        pthread_create(&philosophers[i], &attributes[i], philosopher, (void *)i);
    }

    for (i = 0; i < NUMBER_OF_PHILOSOPHERS; ++i) {
        pthread_join(philosophers[i], NULL);
    }
    return 0;
}
void *philosopher(void * philosopherNumber) {
    while (1) {
        think((long)philosopherNumber);
        pickUp((long)philosopherNumber);
        eat((long)philosopherNumber);
        putDown((long)philosopherNumber);
    }
}

void think(int philosopherNumber) {
    int sleepTime = rand() % 3 + 1;
    printf("Philosopher %d will think for %d seconds\n", philosopherNumber, sleepTime);
    sleep(sleepTime);
}

void pickUp(int philosopherNumber) {
    int right = (philosopherNumber + 1) % NUMBER_OF_PHILOSOPHERS;
    int left = (philosopherNumber + NUMBER_OF_PHILOSOPHERS) % NUMBER_OF_PHILOSOPHERS;

    if (philosopherNumber == 0) {
        // [illegible] 0 [illegible] [illegible] [illegible] [illegible]
        printf("Philosopher %d is waiting to pick up chopstick %d\n", philosopherNumber, right);
        pthread_mutex_lock(&chopsticks[right]);
        printf("Philosopher %d picked up chopstick %d\n", philosopherNumber, right);
        printf("Philosopher %d is waiting to pick up chopstick %d\n", philosopherNumber, left);
        pthread_mutex_lock(&chopsticks[left]);
        printf("Philosopher %d picked up chopstick %d\n", philosopherNumber, left);
    } else {
void eat(int philosopherNumber) {
    int eatTime = rand() % 3 + 1;
    printf("Philosopher %d will eat for %d seconds\n", philosopherNumber, eatTime);
    sleep(eatTime);
}

void putDown(int philosopherNumber) {
    printf("Philosopher %d will put down his chopsticks\n", philosopherNumber);
    pthread_mutex_unlock(&chopsticks[(philosopherNumber + 1) % NUMBER_OF_PHILOSOPHERS]);
    pthread_mutex_unlock(&chopsticks[(philosopherNumber + NUMBER_OF_PHILOSOPHERS) % NUMBER_OF_PHILOSOPHERS]);
}
```

.2



3. הסבר:

הרצנו את הקוד המתוקן מספר פעמים ובדקנו שהפילוסופים ממשיכים לחשוב ולאכול מבלי להיתקע במצב של המתנה לאינסוף.

הפלט המציג את ההתנהגות של הפילוסופים, מראה שכל פילוסוף מצליח להרים את המקלות, לאכול ולהוריד אותם בצורה סדירה.

לאורך זמן הריצה של התכנית, לא חווינו קיפאון והפעולות התבצעו בצורה חלקה.

זה מעיד על כך שהפתרון המתוקן שלנו מונע קיפאון ועובד בצורה תקינה.